

6강

연산자





1. 항과 연산자

- 항(operand) : 연산자에 사용되는 값
 - 연산자(operator) : 항을 이용하여 연산하는 기호
- 항의 개수에 따른 연산자 구분

연산자	설명	연산 예
단항 연산자	항이 한 개인 연산자	<code>++num</code>
이항 연산자	항이 두 개인 연산자	<code>num1 + num2</code>
삼항 연산자	항이 세 개인 연산자	<code>(5 > 3) ? 1 : 0</code>

2. 대입 연산자

- 변수에 값을 대입 하는 연산자
- 연산의 결과를 변수에 대입
- 우선 순위가 가장 낮은 연산자
- 왼쪽 변수(에 오른쪽 변수(값)을 대입

```
int age = 24 // 나이를 의미하는 age 변수에 값 24를 대입(할당)함
```

```
totalScore = mathScore + engScore; // ① mathScore 값과 영어 점수 engScore 값을 더하여  
// ② 총점을 의미하는 totalScore 변수에 대입(할당)함
```

②

①



➤ 대입 연산자 간단한 예제

'='는 수학에서는 '오른쪽 값과 왼쪽 값이 같다'라는 의미이지만, 프로그램에서는 '오른쪽 값을 왼쪽에 대입' 하는 의미로 쓰인다.

프로그램에서 '오른쪽과 왼쪽이 같다' 의미는 '=='이다.

```
public static void main(String[] args) {  
    int x = 10;  
    int y = 20;  
    System.out.println("x=" + x);  
    System.out.println("y=" + y);  
  
    x = y;  
    System.out.println("x=" + x);  
    System.out.println("y=" + y);  
}
```



Console X
<terminated> Test
x=10
y=20
x=20
y=20



3. 산술 연산자

- 피연산자를 이용해서 +, -, *, /, % 등을 수행한다.

+	덧셈
-	뺄셈
*	곱셈
/	나눗셈
%	나머지

```
public static void main(String[] args) {  
    int x = 10;  
    int y = 20;  
    // 산술 연산자.  
    System.out.println("x + y =" + (x+y));  
    System.out.println("x - y =" + (x-y));  
    System.out.println("x * y =" + (x*y));  
    System.out.println("x / y =" + (x/y));  
    System.out.println("x % y =" + (x%y));  
}
```



Console X Pro
<terminated> Test01 [Jav
x + y =30
x - y =-10
x * y =200
x / y =0
x % y =10



4. 증감 연산자

- 1만큼 증가하거나 감소를 수행한다.
- 단항 연산자
- 항의 앞/뒤 위치에 따라 연산의 결과가 달라짐을 유의하자!

연산자	기능	연산 예
++	항의 값에 1을 더한다.	Value = ++num; // 먼저 num 값이 1 증가한 후 value 변수에 대입 Value = num++; // value 변수에 기존 num 값을 먼저 대입한 후 num 값 1 증가
--	항의 값에서 1을 뺀다.	Value = --num; // 먼저 num 값이 1 감소한 후 value 변수에 대입 Value = num--; // value 변수에 기존 num 값을 먼저 대입한 후 num 값 1 감소



➤ 증감 연산자 간단한 예제

```
public static void main(String[] args) {  
    int num = 10;  
    System.out.println("1을 더한 다음 num 값 출력");  
    System.out.println(++num);  
    System.out.println("=====");  
    System.out.println("기존의 num을 출력 후 1을 더함");  
    System.out.println(num++);  
    System.out.println(num);  
    System.out.println("=====");  
}
```



Console × Problems @ Javadoc Declaration
<terminated> Test01 [Java Application] C:\Program Files\Java\jdk-11.0.2\bin\java.exe

```
1을 더한 다음 num 값 출력  
11  
=====  
num을 출력 후 1을 더한 후 출력  
11  
12  
=====
```

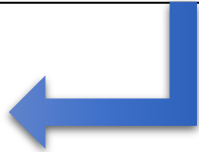


5. 관계 연산자

- 두개의 피연산자를 비교해서 참(True)/거짓(False)의 결론을 도출한다.
- 조건문 if에서 많이 사용한다.

>	a>b : a가 b보다 크면 참
<	a<b : a가 b보다 작으면 참
>=	a>=b : a가 b보다 크거나 같으면 참
<=	a<=b : a가 b보다 작거나 같으면 참
==	a==b : a와 b가 같으면 참
!=	a!=b : a와 b가 같지 않으면 참

```
public static void main(String[] args) {  
    // 관계연산자  
    int x=10;  
    int y=15;  
    System.out.println("x>y:"+x>y);  
    System.out.println("x<y:"+x<y);  
    System.out.println("x>=y:"+x>=y);  
    System.out.println("x<=y:"+x<=y);  
    System.out.println("x!=y:"+x!=y);  
    System.out.println("x==y:"+x==y);  
}
```



```
Console X Problems  
<terminated> Test01 [Java Appli  
x>y:false  
x<y:true  
x>=y:false  
x<=y:true  
x!=y:true  
x==y:false
```



6. 논리 연산자

- 피연산자의 논리곱(AND), 논리합(OR), 논리부정(NOT)을 수행한다.
- 논리 곱(&&)은 두 항이 모두 true 일 때만 결과가 true
=> 앞의 항이 false 이면 뒤 항의 결과를 평가하지 않아도 false 임
- 논리 합(||)은 두 항이 모두 false 일 때만 결과가 false
=> 앞의 항이 true이면 뒤 항의 결과를 평가하지 않아도 true 임
- 조건문 if에서 많이 사용한다.

&& 논리곱(AND)

a && b : a와 b가 모두 참이면 참

|| 논리합(OR)

a || b : a와 b중 하나라도 참이면 참

! 논리부정(NOT)

! a : a의 상태를 부정

```

public static void main(String[] args) {
    int num1 = 10;
    int i=2;
    boolean value = ((num1 = num1+10)<10) && ((i=i+2) < 10);
    System.out.println(value);
    System.out.println(num1);
    System.out.println(i);
    System.out.println("=====");
    value = ((num1 = num1+10)<10) || ((i=i+2) < 10);
    System.out.println(value);
    System.out.println(num1);
    System.out.println(i);
}

```

논리 곱에서 앞의 항의 결과 값이 거짓(false)이므로 실행되지 않는다.

논리 합에서 앞의 항의 결과 값이 참(true)이므로 실행되지 않는다.

Console X Problems

<terminated> Test01 [Java App]

```

false
20
2
=====
true
30
4

```




7. 복합 대입 연산자

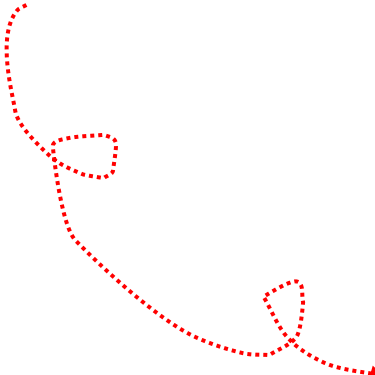
- 대입 연산자와 다른 연산자를 함께 사용한다
- 프로그램에서 자주 사용하는 연산자

연산자	설명	연산 예
<code>+=</code>	두 항의 값을 더해서 왼쪽 항에 대입합니다.	<code>num1 += 2;</code> <code>num1 = num1 + 2;</code> 와 같다.
<code>-=</code>	왼쪽 항에서 오른쪽 항을 빼서 그 값을 왼쪽 항에 대입합니다.	<code>num1 -= 2;</code> <code>um1 = num1 - 2;</code> 와 같다.
<code>*=</code>	두 항의 값을 곱해서 왼쪽 항에 대입합니다.	<code>num1 *= 2;</code> <code>num1 = num1 * 2;</code> 와 같다.
<code>/=</code>	왼쪽 항을 오른쪽 항으로 나누어 그 몫을 왼쪽 항에 대입합니다.	<code>num1 /= 2;</code> <code>num1 = num1 / 2;</code> 와 같다.
<code>%=</code>	왼쪽 항을 오른쪽 항으로 나누어 그 나머지를 왼쪽항에 대입합니다.	<code>num1 %= 2;</code> <code>num1 = num1 % 2;</code> 와 같다.



➤ 복합 대입 연산자 간단한 예제

```
public static void main(String[] args) {  
    // 복합대입 연산자  
    int num1 = 10;  
    System.out.println("num1 += 10 : " + (num1 += 10) );  
    System.out.println("num1 -= 10 : " + (num1 -= 10) );  
    System.out.println("num1 *= 10 : " + (num1 *= 10) );  
    System.out.println("num1 /= 10 : " + (num1 /= 10) );  
    System.out.println("num1 %= 10 : " + (num1 %= 10) );  
}
```



Console X Problems @ Javadoc
<terminated> Test01 [Java Application] C:\WP
num1 += 10 : 20
num1 -= 10 : 10
num1 *= 10 : 100
num1 /= 10 : 10
num1 %= 10 : 0



8. 삼항 연산자

- 삼항 연산자로 두개의 피연산자 연산 결과에 따라서 나머지 피연산자가 결정된다.

조건식 ? 식1 : 식2

조건식이 참이면 식1이 실행되고,
조건식이 거짓이면 식2가 실행된다.

```
public static void main(String[] args) {  
    // 삼항 연산자  
    int x=10, y=20;  
    int result = 0;  
    result = (x > y)? 1000 : 2000;  
    System.out.println("x > y :"+result);  
    result = (x < y)? 1000 : 2000;  
    System.out.println("x < y :"+result);  
    result = (x == y)? 1000 : 2000;  
    System.out.println("x == y :"+result);  
}
```

```
Console X Problems  
<terminated> Test01 [Java App  
x > y :2000  
x < y :1000  
x == y :2000
```



9. 비트 연산자

- 비트연산자는 정수에만 사용 할 수 있다.
- 데이터를 비트(bit)단위로 환산하여 연산을 수행하며, 다른 연산자보다 연산 속도가 향상된다.
- 권한, 상태값 관리에 사용한다.
- 일반 웹 서비스에서는 거의 사용하지 않는다.
- 성능이 민감한 서비스에서는 여전히 사용을 많이 한다. (예 : 게임옵션, 게임엔진, 알림 설정등....)

연산자	설명	예
~	비트의 반전 (1의 보수)	$a = \sim a;$
&	비트 단위 AND	1 & 1 1반환 그 외는 0
	비트 단위 OR	0 0 0반환 그 외는 1
^	비트 단위XOR	두개의 비트가 서로 다른 경우 에 1을 반환
<<	왼쪽 shift	$a \ll 2$ 변수 a를 2비트 만큼 왼쪽으로 이동
>>	오른쪽 shift	$a \gg 2$ 변수 a를 2비트만큼 오 른쪽으로 이동
>>>	오른쪽 shift	>> 동일한 연산 채워지는 비트가 부호와 상관 없이 0 임



9. 비트 연산자

- &(AND) 연산자: 두 비트가 모두 1인 경우만 1 아니면 0

```
int num1 = 5;
int num2 = 10;
int result = num1 & num2;
```



```
num1 : 00000101
& num2 : 00001010
result : 00000000
```

- | (OR) 연산자: 두 비트가 모두 0인 경우만 0 아니면 1

```
int num1 = 5;
int num2 = 10;
int result = num1 | num2;
```



```
num1 : 00000101
| num2 : 00001010
result : 00001111
```

- ^(XOR) 연산자: 두 비트가 다른 값이면 1, 같은 값이면 0

```
int num1 = 5;
int num2 = 10;
int result = num1 ^ num2;
```



```
num1 : 00000101
^ num2 : 00001010
result : 00001111
```

- ~(반전) 연산자: 비트 값을 0은 1로 1은 0으로 바꾸는 연산자

```
int num = 10;
int result = ~num;
```



```
num : 00001010
~ num : 11110101
```



➤ 비트 연산자 간단한 예제

<<(왼쪽 Shift) : 비트를 왼쪽에서 오른쪽으로 이동하는 연산자

=> 비트를 왼쪽으로 한번만 이동하면 $\times 2$ 한 결과

=> 비트를 왼쪽으로 두 번이상 이동하면 $\times 2^n$ 한 결과를 얻을 수 있다.

>>(오른쪽 Shift) : 비트를 오른쪽에서 왼쪽으로 이동하는 연산자

=> 비트를 왼쪽으로 한번만 이동하면 $\div 2$ 한 결과

=> 비트를 왼쪽으로 두 번이상 이동하면 $\div 2^n$ 한 결과를 얻을 수 있다.

정수 15의 왼쪽 2자리 이동는 경우 ($15 \ll 2$)는 경우

00000000 00000000 00000000 00001111



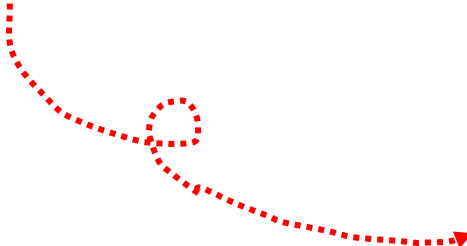
00000000 00000000 00000000 0000111100

→ 결과 : 60



➤ 비트 연산자 간단한 예제

```
public static void main(String[] args) {  
    // 비트 연산자  
    int num = 5; // 00000101  
    // 왼쪽으로 1bit 이동한 경우 결과값은 num값의 2를 곱한 값이 출력된다.  
    System.out.println("num << 1 =>" + (num << 1));  
    // num값은 변경되지 않음을 주의하자!  
    System.out.println(num);  
    // num값을 변경하고 싶다면  
    // System.out.println(num <<= 1);  
    System.out.println("num << 2 =>" + (num << 2));  
    System.out.println("num << 3 =>" + (num << 3));  
    // 00000010 num값을 나누기 2한 값이 출력됨  
    System.out.println("num >> 1 =>" + (num >> 1));  
    System.out.println("num >> 2 =>" + (num >> 2));  
}
```



Console × Problems
<terminated> Test01 [Java Appl
num << 1 =>10
5
num << 2 =>20
num << 3 =>40
num >> 1 =>2
num >> 2 =>1

연습문제





➤ 여기서 잠깐 !

1. 다음 코드의 출력 결과를 쓰시오.

```
int a = 10;  
  
a += 5;  
  
a *= 2;  
  
System.out.println(a);
```

30

1. 다음 코드의 출력 결과를 쓰시오.

```
int x = 5;  
  
System.out.println(x++);  
  
System.out.println(++x);  
  
System.out.println(x);
```

5

7

7



➤ 여기서 잠깐 !

3. 다음 중 출력 결과가 true인 것은?

```
int a = 10;
```

```
int b = 20;
```

```
System.out.println(a > 5 && b < 30);
```

T

```
System.out.println(a > 15 || b < 10);
```

F

4. 다음 코드 실행 시 출력 결과는?

```
int n = 5;
```

```
boolean result = (n > 10) && (++n > 5);
```

F

```
System.out.println(n);
```

5

```
System.out.println(result);
```

F



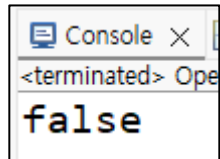
➤ 여기서 잠깐 !

- example패키지안에 OperateEx01.java로 파일을 작성하고 아래 <출력 결과>와 같이 출력되도록 코드를 작성하시오.

조건

- ① 변수와 연산자를 이용하여 작성하시오.
- ② 점수가 60점 이상이면 true 출력 되도록 작성하시오.
- ③ `int score = 30;` 이용해 작성하시오.
- ④ 아래 <출력 결과>와 같이 출력되도록 코드를 작성하시오.

<출력결과>





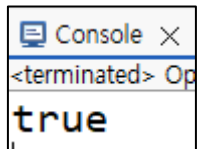
➤ 여기서 잠깐 !

- example패키지안에 OperateEx02.java로 파일을 작성하고 아래 <출력 결과>와 같이 출력되도록 코드를 작성하시오.

조건

- ① 변수와 연산자를 이용하여 작성하시오.
- ② `int money = 178600;` 이용해 작성하시오.
- ③ money 중 5000원짜리 이상 화폐를 제외한 1000원짜리가 3장 이상이면 true가 출력 되도록 작성하시오.

<출력결과>





➤ 여기서 잠깐 !

- example패키지안에 OperateEx03.java로 파일을 작성하고 아래 <출력 결과>와 같이 출력되도록 코드를 작성하시오.

조건

- ① 변수와 연산자를 이용하여 작성하시오.
- ② `int kor = 100; int eng = 87; int math = 41;` 이용해 작성하시오.
- ③ 3과목의 평균이 60점 이상이면, `true` 단, 어느 한 과목이라도 50점 미만이면, `false`가 출력되도록 작성하시오.

<출력결과>

